

Web Programming

Web开发技术基础课程设计

2026年春季学期

 计算机学院（国家示范性软件学院）

 授课人：王尊亮

“Web开发技术基础” 课程设计

课程主题：从AI编程到云端交付 —— 现代Web全栈开发实战

1. AI时代的软件开发

2. 课程设计--**目标**

3. 课程设计--**进度安排**

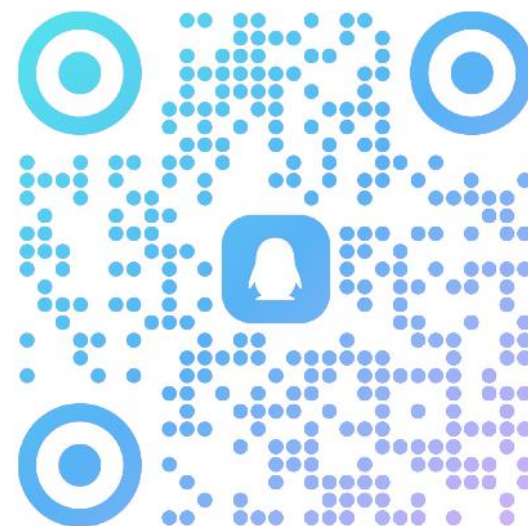
- 阶段一：热身与赋能 —— AI驱动的快速开发
- 阶段二：跨越鸿沟 —— 生产级部署与上线
- **阶段三：实战演练 —— 复杂系统开发挑战**

4. 课程设计--过程要求与评分标准



26Web开发课程设计

群号：1090483732



扫一扫二维码，加入群聊

3. 课程设计--进度安排

- 阶段一：热身与赋能 —— AI驱动的快速开发（1-3周）
 - 内容：匿名树洞/校园表白墙 (Anonymous Wall)
 - 安排：第1周讲解，第2周答疑，第3周**检查**
- 阶段二：跨越鸿沟 —— 生产级部署与上线（4-6周）
 - 安排：第4周讲解，第5周答疑，第6周**检查**
- **阶段三：实战演练 —— 复杂系统开发挑战（7-15周）**
 - 第7周讲解，第8、9答疑，第10-11周**阶段检查**，第12-14周答疑，第15-16周**验收**

6. 阶段三：实战演练 —— 复杂系统开发挑战

题目：校园“万事达”——互助与众包任务平台

项目背景：在快节奏的校园生活中，学生群体中存在着大量零碎且具有时效性的服务需求，如代领快递、代购餐食、搬运重物等；与此同时，也有许多学有余力的学生希望利用碎片时间通过互助服务获取奖励。本项目旨在搭建一个安全、高效的校园众包互助平台，通过引入状态机逻辑、资金托管机制以及信用评价体系，解决校园服务供需匹配的问题。对于开发者而言，该系统将从简单的信息展示跨越到复杂的业务逻辑闭环，是锻炼数据一致性处理与高并发业务建模的理想实战场景。

6. 阶段三：实战演练 —— 复杂系统开发挑战

题目：校园“万事达”——互助与众包任务平台

1. 角色定义

系统需支持以下三种角色，并实现完善的权限隔离（RBAC）：

- 需求方 (Requester): 发布任务（取快递、代买饭、搬东西等），支付报酬。
- 接单方 (Helper): 浏览任务池，接单并完成任务，赚取报酬。
- 管理员 (Admin): 审核违规任务、处理用户申诉、监控平台数据。

6. 阶段三：实战演练 —— 复杂系统开发挑战

题目：校园“万事达”——互助与众包任务平台

2. 核心功能模块要求

A. 任务全生命周期管理 (核心挑战：状态机)

任务必须严格遵循以下状态流转，且状态变更需记录日志：

- 待接单：需求方发布任务后进入。
- 进行中：接单方锁定任务，系统进入“担保”状态。
- 待验收：接单方上传完成凭证（图片/文字）。
- 已完成：需求方确认支付，资金流转。
- 争议中：双方对结果不满意，申请管理员介入。

6. 阶段三：实战演练 —— 复杂系统开发挑战

题目：校园“万事达”——互助与众包任务平台

2. 核心功能模块要求

B. 虚拟账户与资金系统 (核心挑战：事务一致性)

- 余额系统：每个用户拥有虚拟余额。
- 冻结机制：当需求方发布任务时，对应金额需被系统锁定/冻结，防止“余额不足”却已发布任务。
- 结算逻辑：任务完成后，冻结资金转入接单方账户。
- 流水记录：所有的收入、支出、冻结操作必须有对应的流水单据，做到账目可追溯。

6. 阶段三：实战演练 —— 复杂系统开发挑战

题目：校园“万事达”——互助与众包任务平台

2. 核心功能模块要求

C. 实时通知系统 (核心挑战：消息传递)

- 当任务被接单、任务状态变更或收到私聊消息时，相关方需实时收到通知。
- 实现建议：采用 WebSocket 进行站内实时推送，或集成第三方邮件 API 发送通知。

D. 信用与评价体系

- 双向评价：任务结束后，双方互评。
- 信用分模型：初始分 100，根据违约情况（如接单后长时间不操作）和评价结果动态计算。

6. 阶段三：实战演练 —— 复杂系统开发挑战

题目：校园“万事达”——互助与众包任务平台

3. 进阶挑战（选做）

- **消息中心：**实现简单的即时通讯（IM）功能，方便供需双方沟通细节。
- **地理位置功能：**简单的距离计算，优先展示离当前用户更近的任务（GeoHash 存储）。
- **AI 辅助审核：**调用大模型 API 自动识别任务描述中是否包含违规/暴力信息。
- 其它有创新性和技术难度的功能

3. 课程设计--进度安排

- 阶段一：热身与赋能 —— AI驱动的快速开发（1-3周）
 - 内容：匿名树洞/校园表白墙 (Anonymous Wall)
 - 安排：第1周讲解，第2周答疑，第3周检查
- 阶段二：跨越鸿沟 —— 生产级部署与上线（4-6周）
 - 安排：第4周讲解，第5周答疑，第6周检查
- 阶段三：实战演练 —— 复杂系统开发挑战（7-15周）
 - 第7周讲解，第8、9答疑，第10-11周阶段检查，第12-14周答疑，第15-16周验收

4. 课程设计--过程要求与评分标准

过程要求:

1. 原则上2-3人组队，不能单人完成。
2. 使用AI进行设计开发。
3. 设计开发过程中使用云文档进行协作，注意积累文档。
4. 开发过程中需使用git进行源代码管理
5. 阶段1和阶段2提交实验报告
6. 阶段3任务需要需要提供完整的项目文档，包括但不限于：需求分析、总体设计、详细设计、安装部署说明、测试报告以及用户操作手册。
7. 开发语言、框架等技术栈不限。

4. 课程设计--过程要求与评分标准

验收形式：系统讲解与演示+报告，将源代码+作业报告+说明.txt压缩打包后命名为学号+姓名的形式后提交，时间待定

评分标准（**暂定**）：

一、阶段1完成情况（**10分**）

二、阶段2完成情况（**10分**）

三、阶段3作品内容及质量（**40分**）

1. 基本功能完成度 20分

2. 扩展功能完成度 10分

3. 系统方案质量 5分

4. UI/UE质量 5分

四、技术文档（20分）

1. 文档格式及结构 10分

2. 文档内容 10分

五、在设计开发过程中AI的使用情况（10分）

六、团队成员分数评定（10分）

根据团队成员的任务划分和工作量，对团队成员个人分数进行评定

7 AI辅助全栈开发路线图 (更新)

Harness Engineering (驾驭工程) 全面学习指南

核心概念

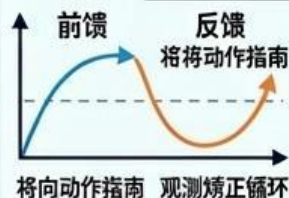


Harness Engineering 是 AI 时代的全新软件工程学科

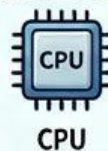
- 约束
- 告知
- 验证
- 修正



核心原理与维度

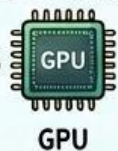


计算型控制



CPU

推理型控制



GPU

三大监管维度



可维护性 Harness

架构适配性 Harness

行为 Harness



技术架构

三大支柱



上下文工程



架构约束

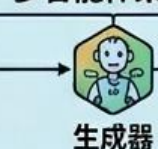


垃圾回收

多智能体架构



规划器



生成器



评估器

最佳实践指南



- ✓ AGENTS.md 作为地图
- ✓ 仓库是唯一真实来源
- ✓ 分层架构强制执行
- ✓ 构建-验证-修复循环
- ✓ 定期垃圾回收

Harness Engineering: 让强大但不可预测的 AI 可靠地完成复杂任务

知乎 @军舰

7 AI辅助全栈开发路线图（更新）

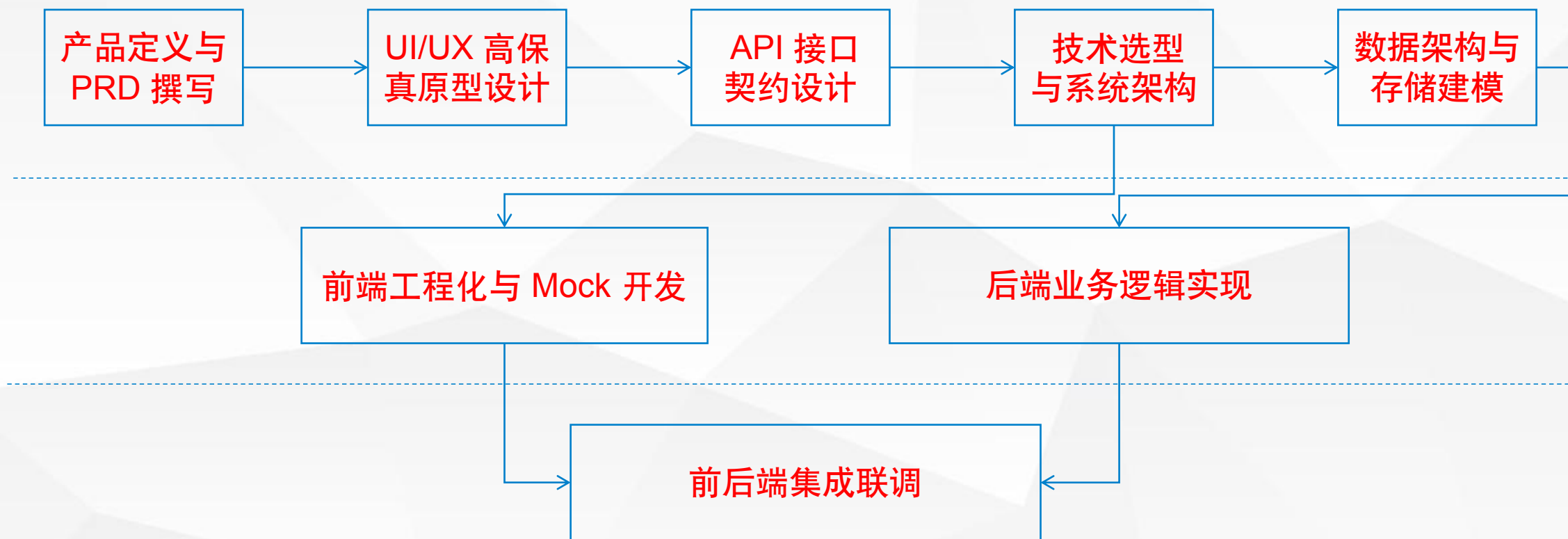
OpenAI 团队在 Harness 工程中的角色转变：

工作内容	传统软件工程	Harness 工程
编写代码	主要工作	从不
设计架构	工作的一部分	主要工作
编写文档	事后考虑	关键基础设施
评审 PR	代码评审	评审智能体输出 + Harness 有效性
调试	阅读代码	分析智能体行为模式
测试	编写测试	设计智能体执行的测试策略

7 AI辅助全栈开发路线图 (更新)

前期发现的问题：对于初学者来说，从抽象的架构图、数据库表结构（E-R图）开始很痛苦。

应对方案：通过“UI 驱动开发 (Design-First Development)”，可以先通过视觉界面理清业务逻辑，再倒推后端实现。



7 AI辅助全栈开发路线图 (更新)

Step 1: 产品定义与PRD 撰写 (Product Definition & PRD)

目标：将模糊的想法具象化为严谨的业务逻辑说明书，形成后续开发的“施工图纸”。

AI 角色：资深产品经理 (Senior PM)

操作步骤：

1. 需求发散：描述核心想法，让 AI 分析市场痛点与用户画像。 (多轮对话修改完善)
2. 构建 PRD：引导 AI 撰写标准 PRD。Prompt 示例：“基于[想法]，请撰写 PRD，需包含角色定义（如需求方、接单方）、核心业务流程、功能模块表（MVP 版本）及业务规则（如抢单冲突处理规则）。” (多轮对话修改完善)
3. 流程可视化：让 AI 使用 Mermaid 语法生成业务状态流转图。 (多轮对话修改完善)

产出物：

- 标准 PRD 文档 (Markdown 格式，含功能矩阵与业务规则)。
- 业务流程图 (Mermaid 代码，清晰描述任务生命周期)。

7 AI辅助全栈开发路线图 (更新)

Step 2: UI/UX 高保真原型设计 (Prototyping)

目标：将 PRD 视觉化，通过界面反向校验功能设计的合理性，建立直观认知。

AI 角色：UI/UX 设计师 / 前端专家

操作步骤：

1. 界面结构设计：利用 AI 工具根据PRD 给出页面风格配色方案、页面清单与布局逻辑的建议。（多轮对话修改完善）。
2. 代码原型生成：利用 AI 工具和前端设计SKILLS根据PRD文档和布局逻辑分模块逐一生成核心页面代码。

Prompt 示例：“请根据 PRD 里的xxx任务模块，使用 Tailwind CSS 生成一个任务列表页高保真原型，布局逻辑为xxx，页面风格为xxx”（多轮对话修改完善）

产出物：

- 高保真 UI 原型代码（HTML/React/Vue 静态片段）。
- 交互说明清单（描述各组件的点击反馈）。

7 AI辅助全栈开发路线图（更新）

🖼️ 预览 全局指挥仓模块设计方案.md ✕

页面定位： 核心监控大屏，用于日常巡查，快速捕捉全球异常。

布局结构： 上下分层，主体分为左侧核心区、右上、右下。

顶部（控制与KPI区）：

全局时间轴控件： 右上角放置一个时间范围选择器（默认“过去30天”）。

4个核心数据指标卡（KPI Cards）：

- 新增事件数（对比上个周期显示 ↑/↓ 红色/绿色箭头）。
- 活跃疾病种数（当前时间段内发生的疾病去重数量）。
- 涉疫国家/地区数。
- 跨物种传播事件数（提取物种既包含动物又包含人类的疾病）。

获取核心指标 KPI (Cards)

后端逻辑： 根据传入的时间范围，执行 COUNT() 和 COUNT(DISTINCT xxx)。返回数据 (data)：

```
{
  "total_alerts": 1250,      // 新增事件数
  "active_diseases": 45,    // 活跃疾病数
  "affected_countries": 12, // 涉疫国家数
  "cross_species_events": 320 // 跨物种事件数
}
```

中部核心区（占屏幕60%）：2D 疫情地理聚合图

- 组件建议： 基于 Mapbox 或 Echarts 的 散点/热力聚合地图。
- 视觉表现： 不要满屏密密麻麻的红点。采用“蜂窝聚合（Hexbin）”或“气泡聚合”。某个国家/省份发生的事件越多，气泡越大、颜色越深（黄->橙->红）。
- 交互： 鼠标悬停（Hover）气泡，弹出 Tooltip（提示框），显示：“所在省份 + 报告事件总数 + Top 3 疾病名称”。

获取地图聚合数据 (Map Aggregation)

接口： GET /api/v1/dashboard/map-nodes

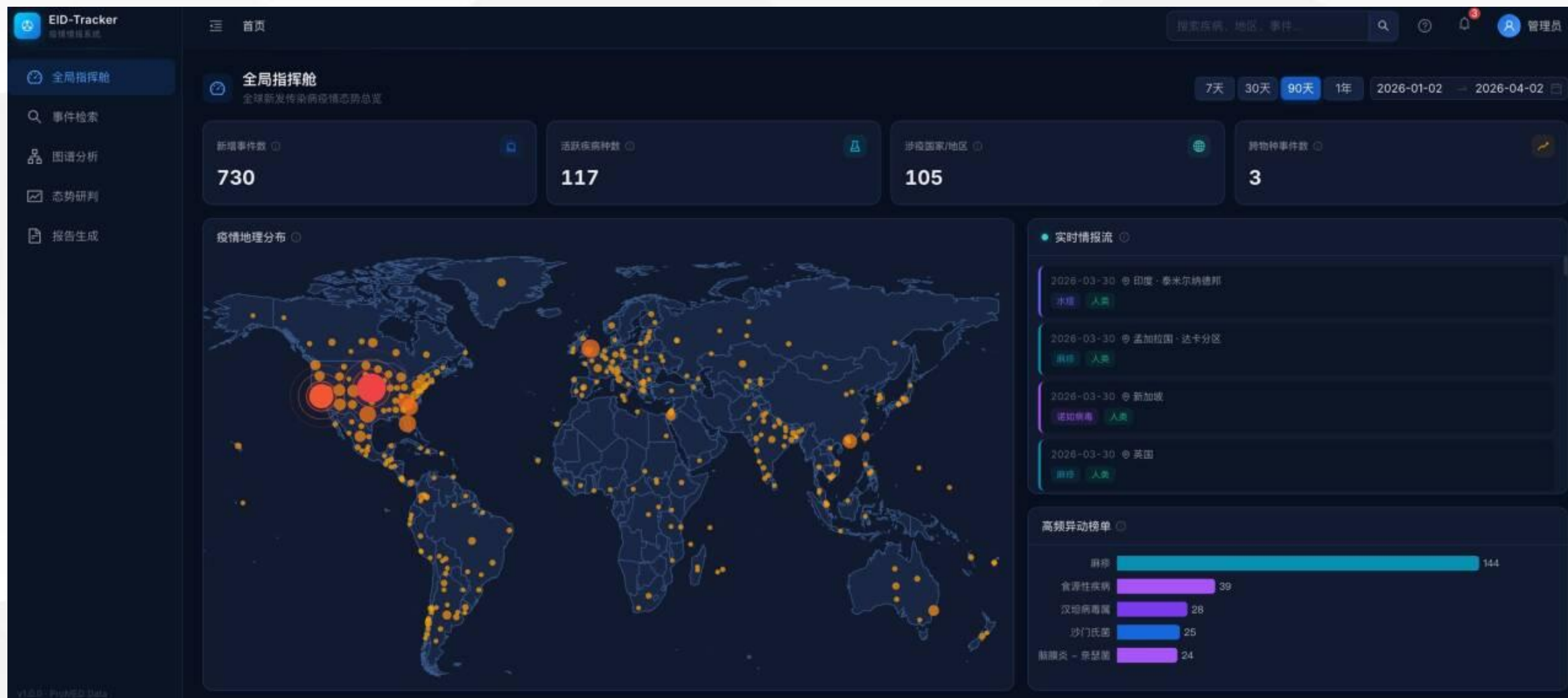
后端逻辑： 绝对不要返回原始散点。按 country/province 进行 GROUP BY，统计每个地区的事件总数，并获取该地区 Top 3 的疾病。

右上区域：实时情报瀑布流（Live Feed）

UI设计： 垂直滚动的 Timeline（时间轴）组件。

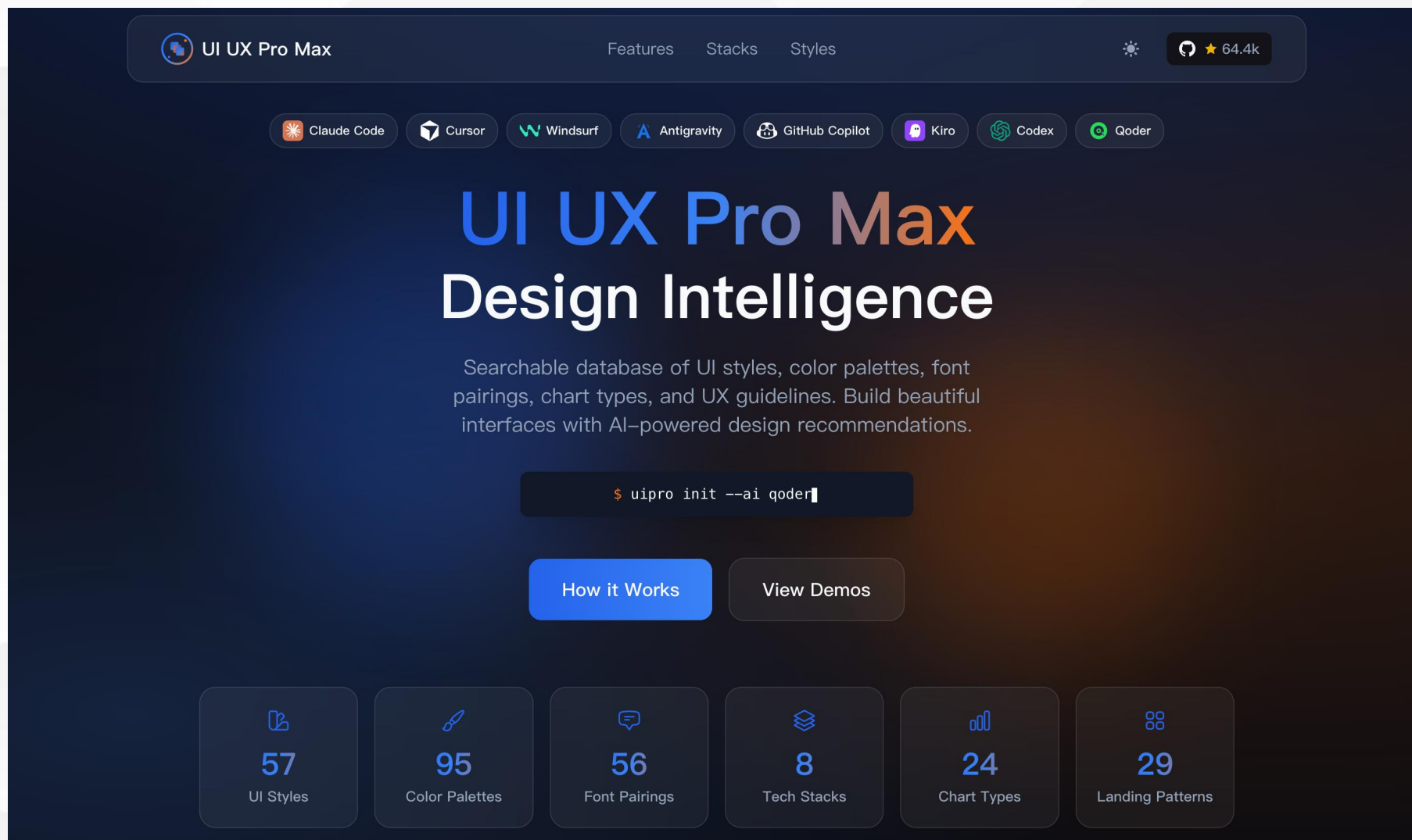
卡片设计： 每一条事件是一个小卡片。 格式： [报告日期] [国家-省] 发生 [疾病名称] 疫情。

7 AI辅助全栈开发路线图 (更新)



7 AI辅助全栈开发路线图 (更新)

<https://ui-ux-pro-max-skill.nextlevelbuilder.io/>



7 AI辅助全栈开发路线图 (更新)

Step 3: API 接口契约设计 (API Design)

目标：既然界面已经有了，那么“界面上的数据从哪来？”“按钮点下去发什么请求？”

AI 角色：后端架构师 / API 专家

操作步骤：

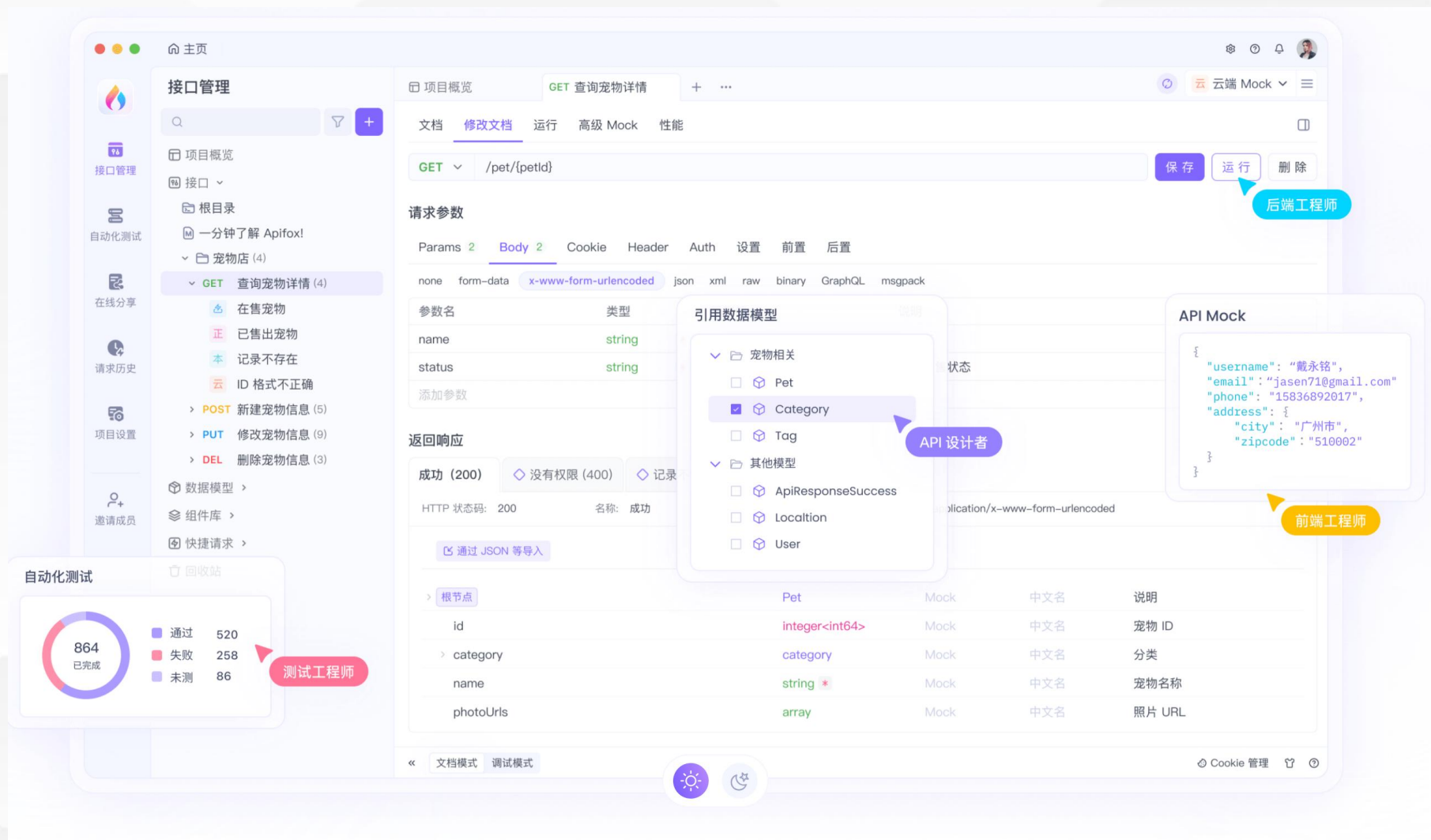
1. 接口推导：把 PRD 和 UI 原型发给 AI，让其分析页面数据流。Prompt 示例：“参考 PRD 和 UI 原型，设计一套 RESTful API。需明确请求路径、方法、Request Body 以及 Response JSON 结构。重点处理‘抢单’接口的返回状态码。” (多轮对话修改完善)
2. 规范对齐：统一字段命名（如 camelCase）、错误码规范及分页逻辑。 (多轮对话修改完善)

产出物：

- API 接口文档（Swagger/OpenAPI 格式或清晰的 Markdown 表格）。

7 AI辅助全栈开发路线图 (更新)

可以将OpenAPI规范的接口文档导入到APIfox等工具中查看



The screenshot displays the APIfox web application interface, which is used for managing and testing APIs. The interface is divided into several sections:

- Left Sidebar:** Contains navigation links for '接口管理' (API Management), '接口' (APIs), '根目录' (Root Directory), '一分钟了解 Apifox!' (Learn Apifox in 1 minute!), '宠物店 (4)' (Pet Shop (4)), 'GET 查询宠物详情 (4)' (GET Query Pet Details (4)), '在售宠物' (Pet for sale), '已卖出宠物' (Pet sold), '记录不存在' (Record does not exist), 'ID 格式不正确' (ID format is incorrect), 'POST 新建宠物信息 (5)' (POST Create pet information (5)), 'PUT 修改宠物信息 (9)' (PUT Modify pet information (9)), 'DEL 删除宠物信息 (3)' (DEL Delete pet information (3)), '数据模型' (Data Model), '组件库' (Component Library), '快捷请求' (Quick Request), '邀请成员' (Invite Members), '自动化测试' (Automated Testing), and '回收站' (Recycle Bin).
- Main Content Area:**
 - Top Bar:** Shows the current project 'GET 查询宠物详情' and tabs for '文档' (Documentation), '修改文档' (Modify Documentation), '运行' (Run), '高级 Mock' (Advanced Mock), and '性能' (Performance).
 - Request Parameters:** Displays the request method 'GET' and the endpoint '/pet/{petId}'. It includes tabs for 'Params', 'Body', 'Cookie', 'Header', 'Auth', '设置' (Settings), '前置' (Pre-request), and '后置' (Post-request). The 'Body' tab is selected, showing the request body in 'x-www-form-urlencoded' format.
 - Response Data:** Shows the response status '成功 (200)' (Success (200)) and the response body in JSON format. It includes a table for the response data with columns for 'id', 'category', 'name', and 'photoUrls'.
 - API Mock:** A section for defining mock data, showing a JSON object with fields like 'username', 'email', 'phone', 'address', 'city', and 'zipcode'.
 - 引用数据模型 (Reference Data Model):** A section for selecting data models to use in the API, showing a list of models including 'Pet', 'Category', 'Tag', 'ApiResponseSuccess', 'Location', and 'User'.
- Bottom Section:** Contains a '自动化测试' (Automated Testing) section with a circular progress indicator showing '864 已完成' (864 Completed). It also includes a table for test results with columns for '通过' (Passed), '失败' (Failed), and '未测' (Not Tested).

7 AI辅助全栈开发路线图（更新）

仓库管理 仓库 CRUD、模板、Domain Schema

GET	/api/isspace/domain-schemas/ 获取Domain Schema定义	get_api_isspace_domain_schemas_	🔒
GET	/api/isspace/repository-templates/ 查询仓库模板列表	get_api_isspace_repository_templates_	🔒
GET	/api/isspace/repository/ 查询仓库列表	get_api_isspace_repository_	🔒
POST	/api/isspace/repository/ 创建仓库	post_api_isspace_repository_	🔒
DELETE	/api/isspace/repository/{id}/ 删除仓库	delete_api_isspace_repository_id__	🔒
GET	/api/isspace/repository/{id}/ 获取仓库详情	get_api_isspace_repository_id__	🔒
PUT	/api/isspace/repository/{id}/ 更新仓库	put_api_isspace_repository_id__	🔒

仓库属性 仓库属性的增删查

GET	/api/isspace/repository/{id}/attributes/ 获取仓库属性	get_api_isspace_repository_id__attributes_	🔒
PUT	/api/isspace/repository/{id}/attributes/ 设置仓库属性	put_api_isspace_repository_id__attributes_	🔒
DELETE	/api/isspace/repository/{id}/attributes/{attr_key}/ 删除仓库属性	delete_api_isspace_repository_id__attributes_attr_key__	🔒

7 AI辅助全栈开发路线图 (更新)

Step 4: 技术选型与系统架构 (Architecture)

目标：确定“地基”与“骨架”，确保系统具备可扩展性和稳定性。

AI 角色：架构师

操作步骤：

AI比较擅长：Python、JavaScript/TypeScript

1. 技术决策：根据 PRD 的复杂度询问 AI 技术选型建议。Prompt 示例：“系统存在抢单高并发场景，建议使用哪种后端框架和缓存策略？请对比 Spring Boot + Redis 与 Node.js 的优劣。” (多轮对话修改完善)
2. 工程结构定义：让 AI 给出标准的项目目录结构。

产出物：

- 技术栈清单（前端、后端、数据库、中间件）。
- 项目分层目录结构图（README 文档形式）。

组件	技术选型
Web 框架	FastAPI (异步)
ORM	SQLAlchemy 2.0 (异步)
数据库	MySQL 8.0 (aiomysql)
数据验证	Pydantic 2
数据迁移	Alembic
主键策略	UUID v7
包管理	uv
认证方式	网关注入 Header

```

i-crms-platform/
├── app/
│   ├── api/                                # API 路由层
│   │   ├── icrms/                          # i-CRMS 模块路由
│   │   │   ├── endpoints/                  # 各功能端点
│   │   │   │   ├── role.py                 # 角色管理
│   │   │   │   ├── menu.py                 # 菜单管理
│   │   │   │   ├── permission.py           # 权限查询
│   │   │   │   ├── platform_role.py        # 平台角色分配
│   │   │   │   ├── project.py              # 项目管理
│   │   │   │   ├── project_permission.py    # 项目成员权限
│   │   │   │   └── user.py                  # 用户相关
│   │   │   ├── router.py                    # 路由聚合
│   │   │   └── __init__.py
│   ├── core/                               # 核心配置
│   │   ├── config.py                       # 应用配置
│   ├── crud/                               # 数据访问层
│   │   ├── base.py                         # CRUD 基类
│   │   ├── association_crud.py             # 关联表操作
│   │   ├── crud_role.py                    # 角色 CRUD
│   │   ├── crud_menu.py                    # 菜单 CRUD
│   │   ├── crud_api_permission.py          # API 权限 CRUD
│   │   ├── crud_project.py                 # 项目 CRUD
│   │   └── ...
│   ├── models/                             # 数据模型层
│   │   ├── base.py                         # 基础模型
│   │   ├── role.py                         # 角色模型
│   │   ├── menu.py                         # 菜单模型
│   │   ├── api_permission.py               # API 权限模型
│   │   ├── project.py                      # 项目模型
│   │   └── ...
│   ├── schemas/                            # Pydantic Schema
│   │   ├── role.py                         # 角色 Schema
│   │   ├── menu.py                         # 菜单 Schema
│   │   └── ...

```

7 AI辅助全栈开发路线图 (更新)

Step 5: 数据架构与存储建模 (Data Architecture & Modeling)

目标：根据 PRD 和 API 需求，设计能够支撑业务逻辑的底层存储结构。

AI 角色：资深数据架构师 / DBA

操作步骤：

1. 提取实体：提取 PRD 中的核心实体 (Entity) 及其属性，并说明它们之间的逻辑关系。 (多轮对话修改完善)
2. 混合存储策略选型：针对不同类型的数据选择最合适的“容器”。 (多轮对话修改完善)
3. E-R 建模：让 AI 将业务实体 (用户、任务、订单) 转化为表结构。 (多轮对话修改完善)
4. 严谨性校验：Prompt 示例：“基于 PRD 业务规则，为我生成 MySQL 建表语句。要求：包含外键约束、索引优化、审计字段 (创建时间等)，并针对‘抢单’逻辑设计乐观锁版本号字段。” (多轮对话修改完善)

产出物：

- 存储策略说明书：明确哪些数据进 SQL，哪些进对象存储，哪些进缓存。
- ER 关系图 (Mermaid)：视觉化展示实体间的逻辑联系。
- SQL 脚本：完整的建表语句，含注释、索引定义和约束。

7 AI辅助全栈开发路线图 (更新)

Step6: 前端工程化与 Mock 开发

目标：在后端未完成前，实现“视觉到功能”的初步转化，保持开发动力。

AI 角色：前端开发工程师

操作步骤：

1. 工程初始化：根据 Step 4 的选型搭建脚手架，创建目录结构。
2. **建立编码契约**：定义技术栈细节，命名规范等。Prompt 示例：“请为我的前端项目制定编码规范。要求：使用 TypeScript 严格模式，组件采用 React 函数式组件 + Hooks，样式使用 Tailwind CSS，网络请求统一封装在 src/api 目录下，并提供一个通用的 Axios 拦截器模板。后续我让你写代码时，请务必遵守这些规范。”
3. **基础骨架与全局布局 (建议SPEC驱动)**：搭建项目的“壳子”，处理路由与全局状态。生成 App.tsx、全局 Layout (导航栏、侧边栏、底部)、配置路由表。Prompt 示例：“根据 Step 4 的目录结构和 Step 2 的 UI 布局，请先写出项目的全局 Layout 组件和 React-Router 基础配置 (包含 首页、任务大厅、个人中心 三个路由路径)。首页、任务大厅、个人中心 三个页面**暂时采用空白占位页面**”

7 AI辅助全栈开发路线图 (更新)

Step6: 前端工程化与 Mock 开发

目标：在后端未完成前，实现“视觉到功能”的初步转化，保持开发动力。

AI 角色：前端开发工程师

操作步骤：

4. 模块化迭代开发 (建议SPEC驱动)： **严禁**让 AI 一次性生成整个 App。要求它按照“组件 -> 页面 -> 逻辑”的顺序迭代。

AI 任务：逐个击破业务模块（如“任务广场模块”、“订单追踪模块”）。

原子化策略：

- UI 组件层：先写原子组件 (TaskCard, StatusBadge) 。
- 页面容器层：将组件组合成页面。
- 数据交互层：注入 Mock 请求逻辑。

Prompt 示例：“现在开始开发‘任务大厅’页面。按照xxx文档中**任务大厅布局逻辑的建议以及** API 文档中的XX章节，设计必要的UI组件、使用 Mock.js 为 相关接口编写模拟数据，然后完成页面开发。”

7 AI辅助全栈开发路线图 (更新)

Step7: 后端业务逻辑实现 (Backend Implementation)

目标：实现从 API 契约到数据库之间的“翻译”和“校验”工作。

AI 角色：后端开发工程师 / 数据库专家

操作步骤：

1. 基础设施与统一脚手架 (Infrastructure & Cross-cutting Concerns)

在写业务前，先定义项目的“公共规则”。

- AI 任务：建立全局异常处理、统一返回格式、身份认证 (JWT/Session) 。
- Prompt 示例：“请根据 Spring Boot (或 FastAPI/Node.js) 规范，为我搭建项目的基础骨架：
 - 定义一个通用的 Result<T> 包装类（包含 code, message, data）；
 - 编写一个全局异常处理器（GlobalExceptionHandler），能够捕获并格式化所有业务异常；
 - 封装一个基础工具类（如日期转换、JWT 校验工具）。”

7 AI辅助全栈开发路线图 (更新)

Step7: 后端业务逻辑实现 (Backend Implementation)

目标：实现从 API 契约到数据库之间的“翻译”和“校验”工作。

AI 角色： 后端开发工程师 / 数据库专家

操作步骤：

2. 持久层映射 (Entity & Persistent Layer) (建议SPEC驱动)

将 Step 5 定义的物理数据库表转换成代码中的逻辑对象。

- AI 任务： 生成 Entity (POJO)、Mapper/Repository 层。
- 操作： 此时数据库已经根据 Step 5 建立，AI 负责根据 SQL 生成实体代码。
- Prompt 示例： “请根据 Step 5 的 SQL 建表语句，生成所有实体类 (Entity) 和对应的 Repository 接口。要求：
： 使用驼峰命名法，实体类需包含对应的数据库字段及注释。”

7 AI辅助全栈开发路线图 (更新)

Step7: 后端业务逻辑实现 (Backend Implementation)

目标：实现从 API 契约到数据库之间的“翻译”和“校验”工作。

AI 角色：后端开发工程师 / 数据库专家

操作步骤：

3. 逐模块垂直开发 (Module-by-Module Sprint) (建议SPEC驱动)

核心策略：每个模块一次性完成“业务逻辑 (Service) + 接口暴露 (Controller)”。建议顺序：用户模块 -> 任务模块 -> 交易模块。

场景示例：开发“任务抢单模块”

- 如使用Qoder的Quest模式，给AI提供“任务抢单模块”的PRD文档，业务流程图、接口文档，数据库设计，让AI编写Spec文档，**开发者澄清需求及审核方案**后执行。

7 AI辅助全栈开发路线图 (更新)

Step7: 后端业务逻辑实现 (Backend Implementation)

目标：实现从 API 契约到数据库之间的“翻译”和“校验”工作。

AI 角色：后端开发工程师 / 数据库专家

操作步骤：

4. 质量守卫：冒烟测试 (Module-wise Testing)

AI 任务：每完成一个模块，生成对应的单元测试。

Prompt 示例：“请为刚才的‘抢单逻辑’编写 JUnit 单元测试，模拟 10 个用户同时请求同一个任务的接口，验证是否只有一个人成功，其他人是否收到了预期的错误提示。”

产出物：

后端源代码（通过 Swagger/Postman 接口测试）。

7 AI辅助全栈开发路线图 (更新)

Step8: 前后端集成联调 (Integration & QA)

目标：拔掉“假数据”，接通“真系统”，完成最后的全链路交付。

AI 角色：全栈工程师 / 测试工程师

操作步骤：

1. 环境对齐：将前端 API 地址指向后端，处理跨域 (CORS) 配置。
2. 链路纠错：运行全流程。遇到报错时，将前端控制台和后端日志同时发给 AI 分析。Prompt 示例：“联调‘支付’接口时前端报 400 错误，后端日志显示参数类型不匹配，请对比接口文档找出问题。”
3. 漏洞扫描：请 AI 分析代码中可能的安全隐患（如 SQL 注入、权限绕过）。

产出物：

- 完整交付的项目源码。
- 测试报告与演示视频。

7 AI辅助全栈开发路线图 (更新)

AI 协作方法论

1. 顶层设计：从“局部模糊”到“全局清晰”，从“我想做”到“AI 能懂”，在动手写代码前，利用 AI 补齐人类容易忽视的**业务边界**。

- 需求锚定 (Requirement Anchoring):

定义角色（谁用）、场景（怎么用）、硬规则（必须遵守什么）、MVP范围（**现在做多少，AI 特别喜欢“过度设计”**，如果你不喊停，它能给你画出一个淘宝的架构。

- 业务规则：需求不仅是“能做什么”，更核心的是“规则是什么”。这些规则是后续写 Service 逻辑时的判断依据。
- 业务流可视化：用 Mermaid 图让 AI 确认它理解的需求路径是否与你一致。。

7 AI辅助全栈开发路线图 (更新)

AI 协作方法论

2. 上下文管理：给 AI 喂“唯一事实之源”

AI 的幻觉通常源于上下文缺失。要建立上下文依赖链：

- 喂入协议 (The Truth)： 在写后端前，先复述一遍 Step 3 定义的 API 契约；在写业务逻辑前，先复述一遍 Step 5 定义的数据库 Schema。
- 状态同步： “当前我们已经完成了基础设施 A 和实体映射 B，现在请开始基于这两个基础实现业务逻辑 C。”
- 模块隔离： 每次对话只处理一个功能闭环。不要试图在一个 Prompt 里让 AI 完成整个系统，那会导致代码质量劣化。

7 AI辅助全栈开发路线图 (更新)

AI 协作方法论

3. 分步实施：从“骨架”到“肌肉”

遵循“由内而外、由点到面”的节奏：

- **基础设施先行** (Infrastructure First)：先把报错格式、返回对象、工具类定死。这相当于给 AI 划定了“装修风格”，后续它生成的代码才不会乱套。
- **垂直切片开发** (Vertical Slicing)：按照“实体 -> 逻辑 -> 接口”的垂直链路逐模块实现。这样可以保证每一个模块写完都是可运行、可测试的。
- **增量演进**：先让 AI 生成“能跑通”的基础版，再通过 Prompt 追加约束（如：“现在请给这段逻辑增加事务支持”、“请优化高并发性能”）。

7 AI辅助全栈开发路线图 (更新)

AI 协作方法论

4. 审核员思维：保持“批判性判断力”

不要当 AI 代码的搬运工，要当**代码审计员**：

- 逻辑校验：重点检查涉及“权限、交易、状态更新”的地方。问 AI：“这段逻辑在并发情况下会产生脏数据吗？”
- 工程质量准则：审查 AI 是否遵循了 Clean Code。例如：是否有硬编码？是否缺少日志记录？是否没有处理资源释放？
- 人工微调：AI 擅长写模版代码，但人类擅长处理“非标准”的特殊业务细节。在 AI 生成的 80% 代码基础上，进行关键 20% 的人工精修。

7 AI辅助全栈开发路线图 (更新)

AI 协作方法论

5. 闭环反馈：以“错误”驱动进化

当代码报错时，不要自己瞎猜，要把 AI 拉进调试流程：

- 错误上下文：将“报错堆栈 + 关联代码 + 数据库状态”一起发给 AI。
- 根因分析：不要只问“怎么修”，要问“为什么会报错”。这样可以发现深层的架构问题（如：依赖冲突或事务失效）。
- 自动化守卫：让 AI 针对它自己写的逻辑编写单元测试。如果测试通过，说明 AI 理解了你的意图。